Design declaration, diagnosis, and redesign | *Déclaration, diagnostic et redesign*

MIDA — introduction | *MIDA — introduction*

Macartan / Alyssa

2026-06-09

Learning goals
Objectifs d'apprentissage

1. Understand what the essential elements of a design are
2. Understand that if you state these elements clearly, you can assess the quality of a design
3. Learn how design adjustments can be optimized
4. Gain exposure to the kind of code that lets you do this in practice

1. Comprendre quels sont les éléments essentiels d'une conception de recherche
2. Comprendre que si vous énoncez clairement ces éléments, vous pouvez évaluer la qualité d'une conception de recherche
3. Apprendre à modifier nos conception de recherche de manière optimale
4. Découvrir le code qui permet de mettre en œuvre ces procédures

Section 1

MIDA framework

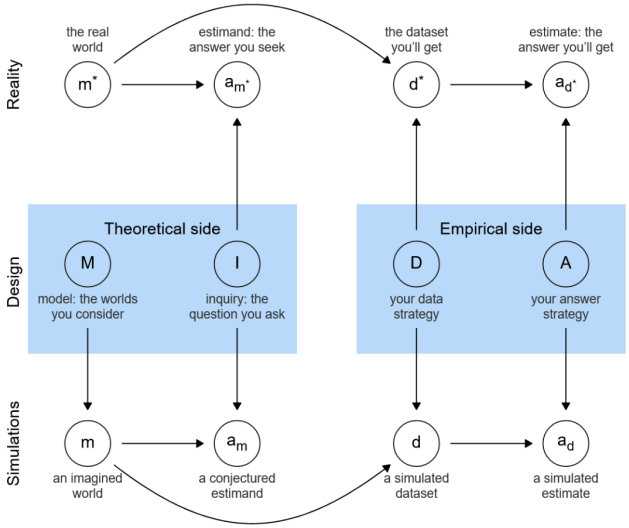
Four elements
Quatre éléments

- **Model**: what causes what, and how
- **Inquiry**: a question stated in terms of the model
- **Data strategy**: sampling, assignment, measurement
- **Answer strategy**: how we summarize the data

- **Model** (*Modèle*) : comment x influence-t-il y (et que sont x et y)
- **Inquiry** (*Interrogation/question de recherche*) : une question formulée selon les termes du modèle
- **Data strategy** (*stratégie en matière de Données*) : échantillonnage, assignation du traitment, mesure
- **Answer strategy** (*stratégie d'Analyse*) : comment on utilise les données pour répondre à la question

MIDA diagram

Schéma MIDA



Declaration
Déclaration

Declaration tells the computer (and readers) what **M**, **I**, **D**, and **A** are.

La **déclaration** indique à l'ordinateur (et aux lecteurs) ce que sont **M**, **I**, **D** et **A**.

Diagnosis
Diagnostic

Diagnosis asks how the design will perform under imagined conditions.
We estimate **diagnosands**: power, bias, RMSE, error rates, ethical harm, amount learned, ...

Le **diagnostic** évalue la performance de la conception de recherche, compte tenu des conditions posées.
On estime des **paramètres de diagnostic** : puissance statistique, biais, erreur quadratique moyenne, taux d'erreur, risques éthiques, quantité apprise, ...

Redesign adjusts data- and answer-strategy features to see how diagnosands change:

- sample size
- randomization procedure
- estimation strategy
- implementation trade-offs (e.g. compliance vs. sample size)

L'étape **redesign** (re-concevoir) adapte la stratégie de données et d'analyse afin d'observer comment les paramètres du diagnostic évoluent. On peut modifier :

- la taille de l'échantillon
- la procédure d'assignation aléatoire (randomisation)
- la stratégie d'analyse
- les compromis liés à la mise en œuvre (p.ex. respect de la conformité vs. la taille d'échantillon)

Section 2

Declaration

Simplest design

design la plus simple

What is the simplest **diagnosable** design?

Quelle est la conception de recherche **diagnostiquable** la plus simple ?

```
simplest_design <-
  declare_model(N = 100, Y0 = rnorm(N), Y1 = .2 + rnorm(N)) +
  declare_inquiry(Q = mean(Y1- Y0)) +
  declare_assignment(X = simple_ra(N)) +
  declare_measurement(Y = X*Y1 + (1-X)*Y0) +
  declare_estimator(Y ~ X)
```

What it does

Ce que fait le code

- M: Draw 100 units with potential outcomes
- I: Define an inquiry: the **sample average effect**
- D: Assign a treatment and measure an outcome
- A: Estimate the mean with a regression intercept

- M : Tirer 100 unités avec leurs résultats potentiels
- I : Définir une interrogation (une question de recherche) : l'**effet moyen dans l'échantillon**
- D : Assigner le traitement et mesurer le résultat
- A : Estimer la moyenne avec l'ordonnée à l'origine d'une régression

The pipe (1)
L'opérateur pipe (1)

Each step is a function that presupposes what earlier steps created.

- Step **order** matters
- Most steps take the `main` data frame in and pass it out

Chaque ligne est une fonction qui s'appuie sur ce qui a été créé lors des étapes précédentes. Le pipe passe le résultat de l'expression de gauche comme premier argument de la fonction de droite.

- L'**ordre** des étapes est important
- La plupart des étapes prennent le cadre de données `main` en entrée et le renvoient en sortie

The pipe (2)
L'opérateur pipe (2)

- `declare_estimator` → `estimator_df`
- `declare_inquiry` → `estimand_df`
- You can run steps **one at a time**

- `declare_estimator` → `estimator_df`
- `declare_inquiry` → `estimand_df`
- Vous pouvez exécuter les étapes **une par une**

Run once

Exécuter le code une fois

Run the whole design once — type its name or call `run_design()`. This produces data, **estimands**, **estimates**, and ancillary statistics.

Exécutez le code de la conception de recherche dans son intégralité une fois — tapez son nom ou appelez `run_design()`. Cela génère des données, des **paramètres**, des **estimations** et des statistiques auxiliaires.

```
simplest_design |> run_design()
```

inquiry	estimand	estimator	term	estimate	std.error	statistic
Q	0.44	estimator	X	0.4	0.21	1.94

Repeat the design many times with `simulate_design()`.

Répétez le design **plusieurs fois** avec `simulate_design()`.

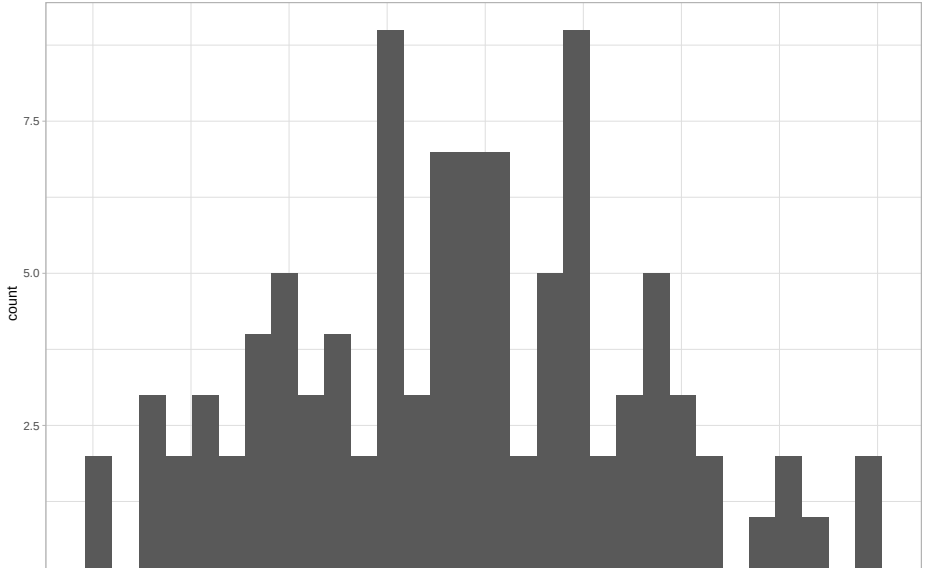
```
some_runs <- simplest_design |> simulate_design(sims = 100)
```

```
some_runs
```

design	sim_ID	inquiry	estimand	estimator	term	estimate
simplest_design	1	Q	0.36	estimator	X	0.39
simplest_design	2	Q	0.47	estimator	X	0.64
simplest_design	3	Q	0.11	estimator	X	0.03
simplest_design	4	Q	0.33	estimator	X	0.60
simplest_design	5	Q	0.13	estimator	X	-0.06
simplest_design	6	Q	-0.18	estimator	X	-0.14
simplest_design	7	Q	0.23	estimator	X	0.04
simplest_design	8	Q	0.38	estimator	X	0.50

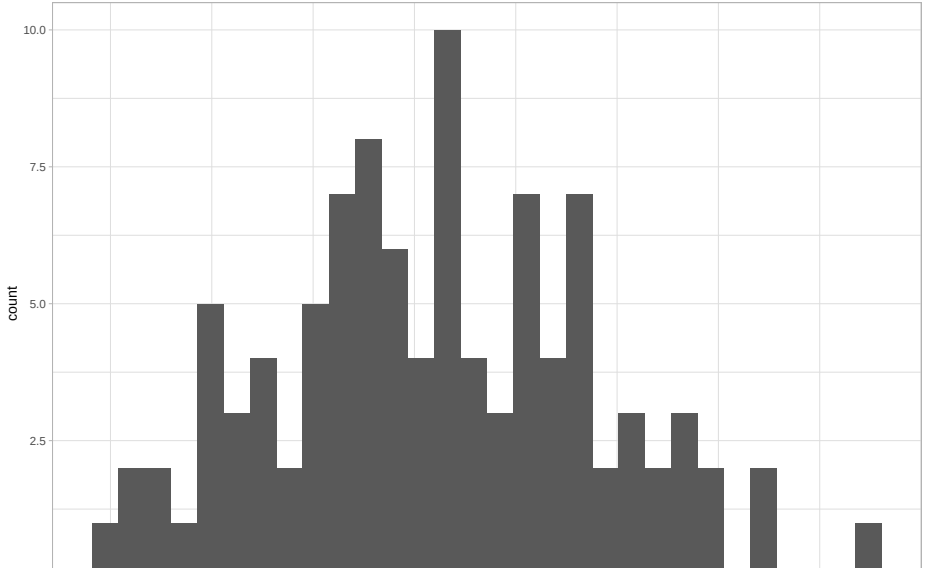
Distribution of estimates

Distribution des estimations



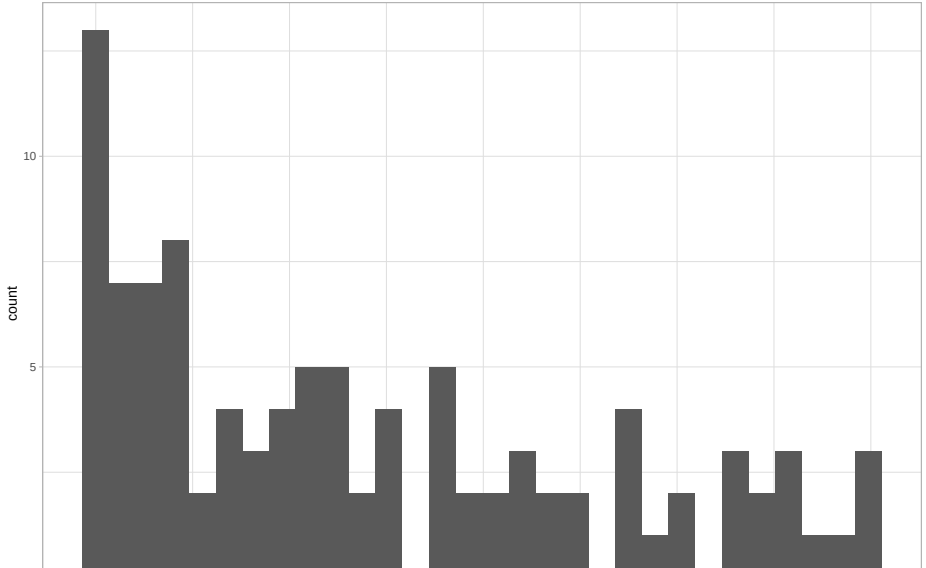
Distribution of errors

Distribution des erreurs



Distribution of p values

Distribution des p-valeurs



Section 3

Diagnosis

Bias by hand

Biais à la main

After many simulations, ask about **bias**: the average gap between estimand and estimate.

Après de nombreuses simulations, mesurez le **biais** : la différence moyenne entre le paramètre et l'estimation.

```
some_runs |>
  mutate(error = estimate - estimand) |>
  summarize(
    mean_estimate = mean(estimate),
    mean_estimand = mean(estimand),
    bias = mean(error)
  )
```

mean_estimate	mean_estimand	bias
0.22	0.2	0.02

`diagnose_design()` computes common diagnosands in one step.

`diagnose_design()` calcule les paramètres de diagnostic en une seule étape.

```
diagnosis <- simplest_design |> diagnose_design()
```

Design	N Sims	Mean Estimand	Mean Estimate	Bias
simplest_design	1000	0.20 (0.00)	0.20 (0.01)	-0.01 (0.00)

Tidy format

Format tidy

Use `tidy()` for a data frame ready to plot.

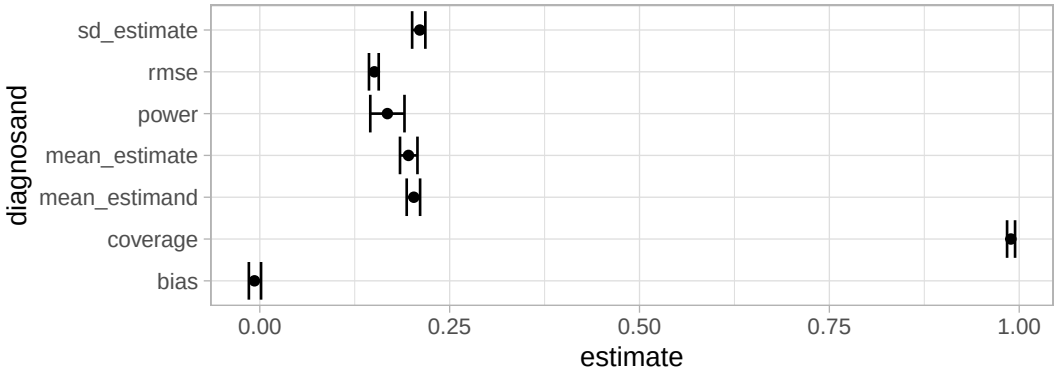
Utilisez `tidy()` pour un data frame prêt à être représenté graphiquement.

```
diagnosis |> tidy()
```

design	inquiry	estimator	outcome	term	diagnosand	e
simplest_design	Q	estimator	Y	X	mean_estimand	
simplest_design	Q	estimator	Y	X	mean_estimate	
simplest_design	Q	estimator	Y	X	bias	
simplest_design	Q	estimator	Y	X	sd_estimate	
simplest_design	Q	estimator	Y	X	rmse	
simplest_design	Q	estimator	Y	X	power	
simplest_design	Q	estimator	Y	X	coverage	

Diagnosis plot

Graphique de diagnostic



Section 4

Redesign

What is redesign?

Qu'est-ce que le redesign ?

Three ways to modify a design:

1. Write new code
2. `replace_step`, `insert_step`, `delete_step`
3. Change a **parameter** with `redesign` ← we focus on this

Trois façons de modifier une conception de recherche :

1. Réécrire du code
2. `replace_step`, `insert_step`, `delete_step`
3. Changer un **paramètre** avec `redesign` ← notre focus

Design parameters
Paramètres de design

A **parameter** is a quantity in your environment that the design references explicitly — e.g. `N` instead of a fixed number.

Un **paramètre** est une quantité dans votre environnement que le design cite explicitement — p.ex. `N` au lieu d'un nombre fixe.

```
N <- 100

simplest_design <-
  declare_model(N = N, Y0 = rnorm(N), Y1 = 1 + rnorm(N)) +
  declare_inquiry(Q = mean(Y1- Y0)) +
  declare_assignment(X = simple_ra(N)) +
  declare_measurement(Y = X*Y1 + (1-X)*Y0) +
  declare_estimator(Y ~ X)
```

Simple redesign
Redesign simple

`redesign()` returns a modified design — here with `N = 200`.

`redesign()` renvoie un design modifié — ici avec `N = 200`.

```
design_200 <- simplest_design |> redesign(N = 200)
design_200 |> draw_data() |> nrow()
```

```
[1] 200
```

A list of designs

Une liste de designs

Pass a **vector** of parameter values to get several designs at once.

Passez un **vecteur** de valeurs de paramètres pour obtenir plusieurs designs d'un coup.

```
design_Ns <- simplest_design |> redesign(N = c(200, 400, 800))
design_Ns |> lapply(draw_data) |> lapply(nrow)
```

```
$design_1
[1] 200

$design_2
[1] 400

$design_3
[1] 800
```

Multiple parameters
Paramètres multiples

With two parameters, la fonction `redesign()` builds a grid of designs — easy to diagnose and compare.

Avec deux paramètres, `redesign()` construit un tableau de designs — facile à diagnostiquer et comparer.

```
N <- 100
b <- .2

simplest_design <-
  declare_model(N = N, Y0 = rnorm(N), Y1 = b + rnorm(N)) +
  declare_inquiry(Q = mean(Y1- Y0)) +
  declare_assignment(X = simple_ra(N)) +
  declare_measurement(Y = X*Y1 + (1-X)*Y0) +
  declare_estimator(Y ~ X)
```

```
designs <- redesign(simplest_design, N = c(100, 200, 500), b = c(0, .2, .5))
designs |> diagnose_design() |> tidy()
```

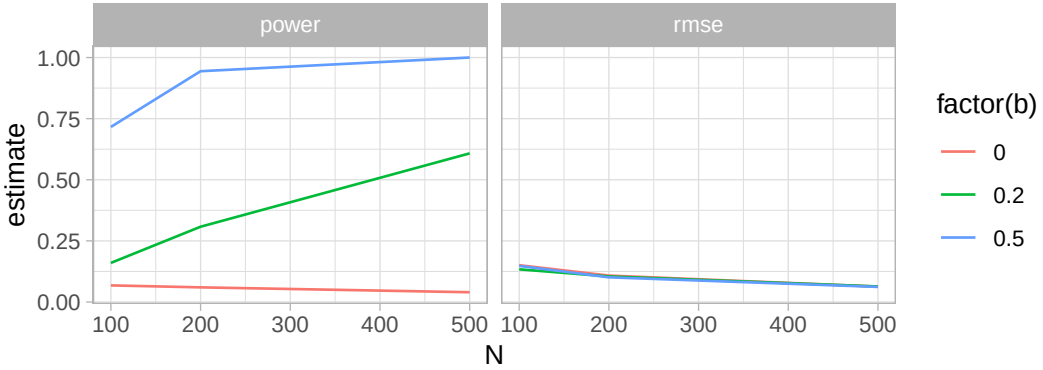
N	b	diagnosand	estimate	std.error	conf.low	conf.high
100	0.0	rmse	0.15	0.01	0.14	0.16
100	0.0	power	0.07	0.02	0.04	0.10
200	0.0	rmse	0.11	0.01	0.10	0.12
200	0.0	power	0.06	0.02	0.03	0.09
500	0.0	rmse	0.06	0.00	0.06	0.07
500	0.0	power	0.04	0.01	0.02	0.07
100	0.2	rmse	0.13	0.01	0.12	0.14
100	0.2	power	0.16	0.02	0.12	0.22
200	0.2	rmse	0.10	0.00	0.10	0.11
200	0.2	power	0.31	0.03	0.26	0.36
500	0.2	rmse	0.06	0.00	0.06	0.07

Plot after redesign

Graphique après redesign

After `tidy()`, ggplot makes comparison across redesigns straightforward.

Après `tidy()`, ggplot permet de comparer facilement les redesigns.



Power depends on N and effect size; RMSE depends on N only.

La puissance statistique dépend du N et de la taille de l'effet ; L'erreur quadratique moyenne ne dépend que de N

Section 5

Resources

declare_* commands

*Commandes declare_*

Key steps for **building** a design:

- `declare_model()`
- `declare_inquiry()`
- `declare_sampling()`
- `declare_assignment()`
- `declare_measurement()`
- `declare_estimator()`
- … and more `declare_*` functions

Étapes clés pour **construire** un design :

- `declare_model()`
- `declare_inquiry()`
- `declare_sampling()`
- `declare_assignment()`
- `declare_measurement()`
- `declare_estimator()`
- … et d'autres fonctions `declare_*`

Using a design
Utiliser un design

Key commands for **running** and **learning** from a design:

- `draw_data(design)`, `draw_estimands(design)`, `draw_estimates(design)`
- `get_estimates(design, data)`
- `run_design(design)`, `simulate_design(design)`
- `diagnose_design(design)`
- `redesign(design, N = 200)`
- `compare_designs()`, `compare_diagnoses()`

Commandes clés pour **exécuter** un design et **en tirer des conclusions** :

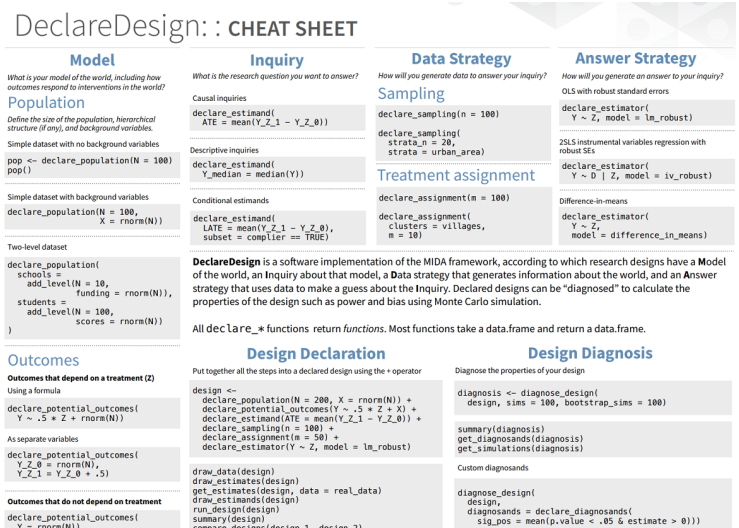
- `draw_data(design)`, `draw_estimands(design)`, `draw_estimates(design)`
- `get_estimates(design, data)`
- `run_design(design)`, `simulate_design(design)`
- `diagnose_design(design)`
- `redesign(design, N = 200)`
- `compare_designs()`, `compare_diagnoses()`

Cheat sheet

Aide-mémoire

RStudio cheat sheet (PDF)

Aide-mémoire RStudio (PDF)



DeclareDesign: : CHEAT SHEET

Model

What is your model of the world, including how outcomes respond to interventions in the world?

Population

Define the size of the population, hierarchical structure (if any), and background variables.

Simple dataset with no background variables

```
pop <- declare_population(N = 100)
pop()
```

Simple dataset with background variables

```
declare_population(N = 100,
                   X = rnorm(N))
```

Two-level dataset

```
declare_population(
  schools =
    add_level(N = 10,
              funding = rnorm(N)),
  students =
    add_level(N = 100,
              scores = rnorm(N))
)
```

Outcomes

Outcomes that depend on a treatment (Z)

Using a formula

```
declare_potential_outcomes(
  Y ~ .5 * Z + rnorm(N))
```

As separate variables

```
declare_potential_outcomes(
  Y_Z_0 = rnorm(N),
  Y_Z_1 = Y_Z_0 + .5)
```

Outcomes that do not depend on treatment

```
declare_potential_outcomes(
```

Inquiry

What is the research question you want to answer?

Causal inquiries

```
declare_estimand(
  ATE = mean(Y_Z_1 - Y_Z_0))
```

Descriptive inquiries

```
declare_estimand(
  Y_median = median(Y))
```

Conditional estimands

```
declare_estimand(
  LATE = mean(Y_Z_1 - Y_Z_0),
  subset = complier == TRUE)
```

Data Strategy

How will you generate data to answer your inquiry?

Sampling

```
declare_sampling(n = 100)
```

```
declare_sampling(
  strata_n = 20,
  strata = urban_area)
```

Treatment assignment

```
declare_assignment(m = 100)
```

```
declare_assignment(
  clusters = villages,
  m = 10)
```

Answer Strategy

How will you generate an answer to your inquiry?

OLS with robust standard errors

```
declare_estimator(
  Y ~ Z, model = lm_robust)
```

2SLS instrumental variables regression with robust SEs

```
declare_estimator(
  Y ~ D | Z, model = iv_robust)
```

Difference-in-means

```
declare_estimator(
  Y ~ Z,
  model = difference_in_means)
```

DeclareDesign is a software implementation of the MIDA framework, according to which research designs have a **M**odel of the world, an **I**nquiry about that model, a **D**ata strategy that generates information about the world, and an **A**nswer strategy that uses data to make a guess about the **I**nquiry. Declared designs can be "diagnosed" to calculate the properties of the design such as power and bias using Monte Carlo simulation.

All `declare_*` functions return *functions*. Most functions take a data.frame and return a data.frame.

Design Declaration

Put together all the steps into a declared design using the + operator

```
design <-
  declare_population(N = 200, X = rnorm(N)) +
  declare_potential_outcomes(Y ~ .5 * Z + X) +
  declare_estimand(ATE = mean(Y_Z_1 - Y_Z_0)) +
  declare_sampling(n = 100) +
  declare_assignment(m = 50) +
  declare_estimator(Y ~ Z, model = lm_robust)
```

```
draw_data(design)
draw_estimates(design)
get_estimates(design, data = real_data)
draw_estimands(design)
run_design(design)
summary(design)
```

Design Diagnosis

Diagnose the properties of your design

```
diagnosis <- diagnose_design(
  design, sims = 100, bootstrap_sims = 100)
```

```
summary(diagnosis)
get_diagnosands(diagnosis)
get_simulations(diagnosis)
```

Custom diagnosands

```
diagnose_design(
  design,
  diagnosands = declare_diagnosands(
    sig_pos = mean(p.value < .05 & estimate > 0)))
```

Other resources

Autres ressources

- Full slide deck: macartan.github.io/dd_bootcamp
- Website: declaredesign.org
- Book: book.declaredesign.org
- Console help: `?DeclareDesign`

- Diapositives complètes : macartan.github.io/dd_bootcamp
- Site web : declaredesign.org
- Livre : book.declaredesign.org
- Aide console : `?DeclareDesign`